NovaTel Day 1 Session 2: Participant Lab (App Service Web Apps)

Oct 13, 2026 · @Nived Varma

In this lab you host the NovaTel Self-Care Portal on Azure App Service, using only the Azure portal in your browser. You create the app, secure it, connect it to a secret in Key Vault, watch its logs, release a new version with zero downtime, and make it scale. Lab 1 follows the trainer's Demo 1 and Lab 2 follows Demo 2.

Before you start

Block	Time	You will
Lab 0: setup	about 10 min	Create a Key Vault and a secret
Lab 1	about 40 min	Create the app, deploy version 1, HTTPS and TLS, app setting, Key Vault reference, logging
Lab 2	45 min	Scale up, staging slot, sticky setting, swap and rollback, autoscale

Your three placeholders. Replace each wherever you see it.

Placeholder	Meaning
`{participantID}`	The ID the trainer gave you, lowercase letters and digits only (for example `p07`). It makes every name unique in the shared subscription.
`{assignedRG}`	The resource group assigned to you. It already exists; you never create one.
`{zipFolder}`	The folder where the trainer put the two code files, portal-v1.zip and portal-v2.zip. You upload them exactly as they are; never unzip them.

Names you will use

Item	Name
Key Vault	`kv-novatel-oc26-{participantID}`
Secret	`CosmosConnectionString`
App Service plan	`asp-novatel-oc26-{participantID}`
Web app	`app-novatel-portal-oc26-{participantID}`
Staging slot	`staging`
Region	The region of your resource group (shown on its Overview page)

Ground rules

- Everything is done in the Azure portal at portal.azure.com. You type nothing into a command window.
- Work only inside your own resource group. Never open or change another participant's resources.
- Create the plan on **Basic B1** in Lab 1. You move to **Standard S1** in Lab 2. Do not pick another size; the subscription is shared and quota is limited.
- If you see a capacity or quota error, tell the trainer. Do not keep retrying.
- Everything you create is deleted by the cleanup script tonight. Nothing is needed tomorrow.
- Stuck for more than 5 minutes? Pair with your neighbour first, then ask the trainer.
- Tip: keep the portal's top search box handy. Typing a resource name there is the fastest way back to it.

Lab 0: setup (about 10 min)

Goal: a Key Vault holding one secret. Later the app will read that secret without any password. In a real project this secret would be the database connection string.

Step 0.1: find your resource group

1. Sign in at portal.azure.com with the account the trainer gave you.
2. In the top search box type `Resource groups` and open it.

3. Open `{assignedRG}`. On its Overview page note the **Location**; you use this region for everything you create.

Checkpoint: you can see your resource group's Overview page and its location. Leave this tab open; it is your home base.

Step 0.2: create the Key Vault

1. In your resource group, click **Create**. In the search box type `Key Vault`, choose **Key Vault**, then **Create**.
2. On the **Basics** tab set:
   - Subscription: the shared training subscription
   - Resource group: `{assignedRG}`
   - Key vault name: `kv-novatel-oc26-{participantID}`
   - Region: the same as your resource group
   - Pricing tier: Standard
   - Days to retain deleted vaults: 7
   - Purge protection: Disable purge protection
3. Click **Next** to **Access configuration**. Set Permission model to **Vault access policy**. Leave everything else as it is.
4. Click **Next** to **Networking**. Keep **Enable public access** with **all networks** selected.
5. Click **Review + create**, then **Create**. Wait for "Your deployment is complete" and click **Go to resource**.

Checkpoint: you are on your Key Vault's Overview page. If the name is rejected as already taken, add a digit at the end and use that name everywhere below.

Step 0.3: add a secret

1. In the left menu of the Key Vault open **Objects**, then **Secrets**.
2. Click **Generate/Import**. Upload options: **Manual**. Name: `CosmosConnectionString`. Secret value: `AccountEndpoint=https://demo.invalid;AccountKey=dummy`.
3. Leave the rest as default and click **Create**.

If you see "You are unauthorized to view these contents", give yourself permission first: in the left menu open **Access policies**, click **Create**, tick **Get**, **List** and **Set** under Secret permissions, click **Next**, search for your own account as the principal, click **Next** twice, then **Create**. Then repeat step 2.

Checkpoint: the Secrets list shows `CosmosConnectionString` with status Enabled.

Lab 1: create, secure, configure, log (about 40 min)

Goal: a live portal app on Basic B1 that forces HTTPS, reads one plain setting and one Key Vault secret, and shows its errors in the log stream.

Step 1.1: create the plan and the web app (7 min)

1. In the top search box type `App Services`, open it, and click **Create**, then **Web App**.
2. On the **Basics** tab set:
   - Resource group: `{assignedRG}`
   - Name: `app-novatel-portal-oc26-{participantID}`
   - Publish: **Code**
   - Runtime stack: **.NET 8 (LTS)**
   - Operating System: **Linux**
   - Region: the same as your resource group
   - Linux Plan: click **Create new** and name it `asp-novatel-oc26-{participantID}`
   - Pricing plan: click **Explore pricing plans**, open the **Dev/Test** tab, select **Basic B1**, then **Select**
   - Zone redundancy: **Disabled**
3. Skip the other tabs (Deployment, Networking, Monitor + secure, Tags). Click **Review + create**, then **Create**.
4. When the deployment completes, click **Go to resource**.

Checkpoint: your resource group now holds two new resources, an App Service plan and an App Service. Open the plan and note the pricing plan (B1) and the instance count (1). On the app's Overview page click the **Default domain** link; you see Azure's default placeholder page. The plan is the compute you pay for; the app is the software that runs on it.

Step 1.2: upload version 1 of the portal (8 min)

1. In your app's left menu open **Deployment**, then **Deployment Center**.
2. At the top choose **Manual Deployment (Push)**. Set **Source** to **Publish files (new)**.
3. Under **File**, browse to `{zipFolder}` and select **portal-v1.zip**. Do not unzip it. The runtime shows .NET 8; leave **Skip Server-Side Build (Pre Built App)** switched off.

4. Click **Deploy**. The upload and build take about 2 to 3 minutes. Open the **Logs** tab and wait until your deployment shows **Success**.

Checkpoint: Deployment Center's Logs tab shows your deployment as **Success**. Browse your app; the page reads **NovaTel Portal v1**. The app also has these pages: `/health` (returns OK), `/fail` (logs an error and returns a 500), `/config` (shows the `PortalEnv` setting and whether the Key Vault secret resolved, never its value) and `/burn` (uses CPU, for the autoscale step). The first load can take 20 to 30 seconds.

Step 1.3: force HTTPS, set the TLS version, turn on Always On (5 min)

1. In your app's left menu open **Settings**, then **Configuration**, then the **General settings** tab.
2. Turn **Always on** to On.
3. Turn **HTTPS Only** to On.
4. Set **Minimum Inbound TLS Version** to **1.2**.
5. Click **Save**, then **Continue** to confirm the restart.

Checkpoint: in a new browser tab type your app's address with `http://` at the start. The address changes to `https://` by itself.

Think: why does Always on matter for a customer portal? What would a customer see without it after a quiet night?

Step 1.4: add an app setting (4 min)

1. In your app open **Settings**, then **Environment variables**, then the **App settings** tab.
2. Click **Add**. Name: `PortalEnv`. Value: `Lab`. Leave **Deployment slot setting** unticked for now. Click **Apply**.
3. Click **Apply** at the bottom of the page, then **Confirm**.

Checkpoint: browse `/config` on your app. It shows `PortalEnv: Lab`. The app reads the setting as an environment variable.

Step 1.5: read a secret from Key Vault without a password (9 min)

The app proves who it is with a managed identity. Key Vault checks that identity and releases the secret. No secret is stored in the app.

1. **Give the app an identity.** In your app open **Settings**, then **Identity**, then the **System assigned** tab. Set Status to **On**, click **Save**, then **Yes**.

2. **Let that identity read secrets.** Open your Key Vault, then **Access policies**, then **Create**. Under Secret permissions tick **Get** only. Click **Next**. In the Principal box search for `app-novatel-portal-oc26-{participantID}` and select it. Click **Next** twice, then **Create**.
3. **Point the app at the secret.** Back in your app open **Environment variables**, then **App settings**, then **Add**. Name: `CosmosConnectionString`. Value:

```
@Microsoft.KeyVault(VaultName=kv-novatel-oc26-
{participantID};SecretName=CosmosConnectionString)
```

Then click **Apply**, then **Apply** at the bottom of the page, then **Confirm**.

Verify it in the portal. Refresh the Environment variables page. Next to `CosmosConnectionString` you see a green tick and the status **Resolved**. If it shows an error, wait a minute and refresh again.

Checkpoint: browse `/config`. It reports `CosmosConnectionString: resolved`. It never prints the secret itself.

Think: if the reference could not be resolved, the app would receive the literal text `@Microsoft.KeyVault(...)` as its value. Where would you look first?

Step 1.6: logging and the log stream (5 min)

1. In your app open **Monitoring**, then **App Service logs**.
2. Set **Application logging** to **File System** with Level **Information**. Set **Web server logging** to **File system**. Click **Save**.
3. Open **Monitoring**, then **Log stream**. Leave this tab open.
4. In another browser tab open `/fail` on your app, and refresh it three times.

Checkpoint: within a few seconds the Log stream shows the error written by the app (`Payment failed for customer 1042`) and the three 500 responses. Streaming can lag, and lines can arrive out of order.

Then open **Diagnose and solve problems** in the left menu and look at the **Availability and Performance** tiles. This is the first place to look when an app misbehaves.

Lab 1 checklist

- [ ] Plan and app exist in my resource group, and the plan is Basic B1
- [ ] `http://` redirects to `https://`; minimum TLS is 1.2; Always on is On

- [ ] `/config` shows `PortalEnv: Lab` and the secret as resolved
- [ ] I saw my `/fail` error in the log stream

Lab 2: scale, release safely, autoscale (45 min)

Goal: release a new version of the portal with no failed requests, roll it back in one step, and set up scaling rules. Start from your finished Lab 1 app.

Step 2.1: hit the Basic limit, then scale up (4 min)

1. Open your app, then **Deployment**, then **Deployment slots**. On Basic you see a message that slots need a higher pricing plan.
2. On the app's **Overview** page click the **App Service plan** link to open your plan. In the plan open **Settings**, then **Scale out (App Service plan)**. Only manual scaling is offered; there is no autoscale.
3. In the plan open **Settings**, then **Scale up (App Service plan)**. On the **Production** tab choose **Standard S1** and click **Select**.

Checkpoint: the change applies in seconds and your app stays up (keep it open in another tab). Reopen **Deployment slots** and **Scale out**; both are now usable. The pricing plan unlocked the features, with the same code and the same address.

Step 2.2: create a staging slot and upload version 2 (12 min)

1. In your app open **Deployment**, then **Deployment slots**, then **Add slot**. Name: `staging`. Clone settings from: your production app. Click **Add**, then **Close**.
2. Click the `staging` slot in the list. You are now looking at a second app with its own address. In its left menu open **Deployment**, then **Deployment Center**. The page title must show your app name followed by `staging`. If it does not, you are on production; go back and open the slot.
3. Choose **Manual Deployment (Push)**. Set **Source** to **Publish files (new)**. Under **File**, select **portal-v2.zip** from `{zipFolder}`. Do not unzip it. Leave **Skip Server-Side Build (Pre Built App)** switched off. Click **Deploy**.
4. Open the **Logs** tab and wait until the deployment shows **Success**. It takes about 2 to 3 minutes.

Checkpoint: on the staging slot's Overview page click its **Default domain** link. It reads **NovaTel Portal v2**. Your production address still reads **NovaTel Portal v1**. Keep both pages open side by side.

The slot runs on the same plan, so staging uses production's capacity. Slots cost nothing extra.

Step 2.3: sticky settings and the slot's own identity (10 min)

A swap moves an app's settings along with it, unless you mark them as slot settings. A setting that points at a database must never travel to the wrong environment.

1. **Production.** Open your app, then **Settings**, then **Environment variables**, then **App settings**. Click `PortalEnv`, tick **Deployment slot setting**, click **Apply**, then **Apply** at the bottom, then **Confirm**.
2. **Staging.** Open the `staging` slot, then **Environment variables**, then **App settings**. Click `PortalEnv`, change the Value to `Staging`, tick **Deployment slot setting**, click **Apply**, then **Apply** at the bottom, then **Confirm**.
3. **Staging's identity.** Each slot has its own managed identity, so staging cannot read Key Vault yet. In the staging slot open **Settings**, then **Identity**, then **System assigned**. Set Status to **On**, click **Save**, then **Yes**.
4. **Let staging read the secret.** In your Key Vault open **Access policies**, then **Create**. Tick **Get** under Secret permissions and click **Next**. In the Principal box search for your app name and choose the entry that ends with `/slots/staging`. Click **Next** twice, then **Create**.

Checkpoint: browse `/config` on the production address; it says `PortalEnv: Lab`. Browse `/config` on the staging address; it says `PortalEnv: Staging`. Both show the secret as resolved. If staging shows unresolved, wait a minute and refresh.

Step 2.4: swap with live traffic, then roll back (10 min)

1. Open your production address with `/health` at the end in a browser tab. It shows OK. You will refresh this tab during the swap.
2. In your app open **Deployment slots** and click **Swap**. Source: `staging`. Target: `production`. The panel lists the settings changes the swap will make; confirm `PortalEnv` is not in the list because it is sticky.
3. Click **Start swap**. While it runs (about a minute), keep refreshing the `/health` tab.

Checkpoint: `/health` shows OK on every refresh, never an error. When the swap completes, production reads **NovaTel Portal v2** and `/config` still shows `PortalEnv: Lab`. The staging slot now holds version 1.

4. **Roll back.** Click **Swap** again with the same source and target and start it. Production returns to **NovaTel Portal v1**. The old version was waiting in the staging slot all along.

A swap is not instant: App Service restarts and warms up the staging instances first, and cancels the swap if the warm-up fails. That warm-up is why users see no downtime.

Step 2.5: autoscale (10 min)

Rules come in pairs: add an instance when CPU is high, remove one when it is low. Scale out fires if any scale-out rule is met; scale in needs all scale-in rules to be met.

1. Open your **plan**, then **Settings**, then **Scale out (App Service plan)**. Choose **Custom autoscale**. Scale mode: **Scale based on a metric**. Instance limits: Minimum `1`, Maximum `3`, Default `1`.
2. Click **Add a rule**. Metric: **CPU Percentage**. Operator: **Greater than**. Threshold: `70`. Duration: `5` minutes. Time grain statistic: **Average**. Action: **Increase count by** `1`. Cool down: `5` minutes. Click **Add**.
3. Click **Add a rule** again. Metric: **CPU Percentage**. Operator: **Less than**. Threshold: `30`. Duration: `5` minutes. Action: **Decrease count by** `1`. Cool down: `5` minutes. Click **Add**.
4. Click **Save**.

CPU rules need sustained load to trigger, so the trainer demonstrates them live. To see scaling happen on your own app, use a schedule, the way NovaTel would plan for month-end:

5. Under the default condition click **Add a scale condition**. Name: `Month-end`. Scale mode: **Scale to a specific instance count**. Instance count: `2`. Schedule: **Specify start/end dates**. Start: today, 5 minutes from now. End: today, 20 minutes from now. Use the portal's time zone. Click **Save**.
6. After the start time, return to the **Scale out** page of your plan and open the **Run history** tab. It shows the instance count changing to 2; this can take a few minutes. You can also open **Monitoring**, then **Metrics**, and chart **CPU Percentage** to see the load on the plan.

Checkpoint: your autoscale setting shows the default condition with two rules and the `Month-end` condition. After the start time the instance count becomes 2 (it can take a few minutes). Then delete the `Month-end` condition and click **Save**, so you do not keep extra instances in the shared subscription.

Think: NovaTel's busiest days are known in advance. Which kind of rule covers the known spike, and which covers the surprise?

Lab 2 checklist

- [ ] The plan is Standard S1 and the `staging` slot exists
- [ ] `PortalEnv` is a slot setting on both slots with different values
- [ ] I swapped to v2 with no errors on `/health`, then swapped back to v1
- [ ] My autoscale has a scale-out rule, a scale-in rule, min 1, max 3, and I removed the `Month-end` condition afterwards

If something goes wrong

Symptom	Likely cause	What to do
Create fails with a quota or capacity error	Shared subscription limit for that size or region	Stop and tell the trainer; do not keep retrying
Key Vault name rejected	Name already taken, or a deleted vault still holds it	Add a digit to the name and use it everywhere
"You are unauthorized to view these contents" in Key Vault	Your account has no secret permissions	Add an access policy for yourself (Step 0.3)
Deployment Center shows Failed, or the app still shows the placeholder page	The zip was unzipped or is the wrong file, or the build failed	Upload the original zip again (do not unzip it), read the error on the Logs tab, then tell the trainer
App is slow on first load	Container still starting	Wait 30 seconds and refresh
Key Vault setting shows an error, or `/config` prints the raw `@Microsoft.KeyVault` text	Access policy not applied yet, wrong vault or secret name, identity not turned on	Wait a minute and refresh; recheck the names; confirm the identity is On and the policy has your app
Staging shows the secret as unresolved	The slot has its own identity	Repeat Step 2.3, parts 3 and 4
Log stream is empty	Logging not saved, or nothing logged yet	Confirm Step 1.6 was saved, open `/fail` again, wait 10 seconds

Symptom	Likely cause	What to do
Deployment slots still says to upgrade	Browser cache	Refresh the page; confirm the plan shows S1
`/health` shows an error during the swap	Slow warm-up	Tell the trainer; note the time
The instance count does not change	The schedule has not started, or the rule window has not passed	Check the start time and time zone; wait a few minutes

Stretch tasks (if you finish early)

- Before swapping, predict what `/config` will show on each slot afterwards, then check.
- On your plan's autoscale, add a second scale-out rule on **Memory Percentage** greater than 75. Predict what happens if CPU is 40 percent and memory is 80 percent.
- Open **Diagnose and solve problems** on your app and find the 500 responses from `/fail`.
- Open the plan's **Activity log** and find the entries for your autoscale changes.
- Container objective (not part of today's hands-on): in your app open **Deployment Center** and look at the **Container registry** option. A container build is covered on Day 2.

Cleanup

Nothing to do. The end-of-day cleanup script deletes the resources in your resource group, including the plan, app, slot, autoscale setting and Key Vault. Before you leave, check that you removed the `Month-end` autoscale condition. Do not delete or edit resources outside your group.